先問取出順序，再決定表示法

Stack 與 queue 都保存一串值，差別是「下一個能取出誰」。本章先建立操作契約，再追蹤索引與物件，最後用同一套狀態思考搜尋及運算式。

學習順序	要能自己回答的問題
堆疊與陣列	top 指向哪裡？擴容為何不是每次都很貴？
環狀佇列	實體位置繞回後，FIFO 次序如何保持？
資料結構互作	兩個 LIFO 如何合成 FIFO？成本如何攤銷？
搜尋	回溯時記住什麼？BFS 為何保證最少邊數？
運算式	運算子何時輸出？左右運算元如何區分？

前置知識：陣列、半開區間與基本迴圈不變量。程式使用的空／滿政策與時間成本，會在各節明訂。

3.1　同一端進出：堆疊的操作契約

- 抽象狀態為序列 $[a_0,\ldots,a_{n-1}]$，頂端在末端。`push(x)` 把 `x` 接在末端。
- `top()` 回傳最後一個元素但不改狀態。`pop()` 移除末端元素，前提是非空。
- 在空堆疊呼叫 `pop` 的行為由介面定義。可回傳 optional、例外或錯誤碼，不能默默讀取無效記憶體。
- 表示為動態陣列時，`top`、`pop` 為常數時間，`push` 為攤銷常數時間。固定容量版本另有滿溢結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

push 與 pop：先看得見狀態

容量4；索引由0起算，陰影區是活物件



序列均由**底端寫到頂端**。`top()` 只讀取 3；`pop()` 移除 3 後，新頂端才變成 8。沒有元素時，不能計算並讀取 `A[n-1]`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用元素數 n，避免空堆疊的負索引

- 不變量為 $0 \le n \le capacity$，有效元素位於 $[0,n)$。頂端是 $A[n-1]$，只有 $n>0$ 才能存取。
- $push(x)$ 先確保有容量，再寫 $A[n]$，最後增加 n。若配置失敗，不應把 n 提前增加。
- pop 先檢查 $n>0$，再移除最後一個有效物件。泛型 C++ 實作還要處理析構，不只是改索引。
- 以 $n=0$ 表示空狀態可避免以無號索引表示 $top=-1$ 的問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

配置空間，不等於建構物件

設 storage 是容量 C 的未初始化儲存，n 是活物件數。抽象序列等於 storage[0..n)；不能把「配置了 C 格」誤當成「已建構 C 個 T」。

```
top(): require n>0; return const T& to storage[n-1]
push(T value):            // snapshot constructed before entering the body
    if n==C: grow_to(max(1,2*C))
    construct_at(storage+n, value)   // 成功後才提交 n
    n=n+1
pop():
    require n>0
    destroy_at(storage+n-1)
    n=n-1
```

https://g

擴容時，究竟搬了多少值？



第三次 push 是線性時間；**攤銷常數不代表每次最壞常數**。本例 pop 不縮容。

push(8)、push(3)、push(2)、pop() 的抽象序列依序為 [8]、[8,3]、[8,3,2]、[8,3]。

倍增成本：把整串操作加起來

從空容量開始，滿時依序擴為 1、2、4、8……。對 m≥1 次 push，最後容量 C 是不小於 m 的最小二次冪，因此 m≤C<2m。

$$\text{搬移次數} = 1 + 2 + 4 + \cdots + C/2 = C - 1 < 2m.$$

另有 m 次新值建構，所以總工作量為 Θ(m)，每次 push 攤銷 Θ(1)。這項結論假設每次元素建構／搬移有固定成本；若 T 是長字串或大矩陣，必須另計值操作成本。

若要混合 pop，這個**不縮容版本**仍可把每個 pop 的常數工作直接加上。若一滿就倍增、一半空就立即減半，可能反覆搬移；縮容門檻必須另外設計與證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

push(top())：先保存值，再搬家

容量 1、內容 [7] 時，參數若是 `const T&` 且直接指向舊 top，擴容釋放舊儲存後，這個參照便失效。`noexcept` 不會讓它重新有效。

本章的按值介面 push(T value)	狀態與理由
進入函式前複製參數	value=7，已是獨立物件
成功擴容並搬入原值	新容量2，舊有序列仍為 [7]
從 value 建構新元素	新物件成功後才把 n 改為2
操作完成	[7,7]；舊儲存的參照不再可用

這裡限定 T 可複製建構，且參數快照的成本也計入呼叫。不可複製的 T 不支援這個複製式 `push(top())`。

例外保證：失敗發生在哪個階段？

失敗點	本節偽碼能保證什麼？
按值參數複製失敗	尚未進入本體；n、capacity 與既有值不變
擴容搬移失敗	要還原原狀，需合適的複製或 noexcept move 策略與清理；不能默認任意 T 都可做到
擴容成功後，新元素建構失敗	n 不增加、原值仍有效；capacity 與舊參照可能已改變

不能把「size 沒增加」說成整個 push 具有強例外保證。 強保證要求呼叫失敗時可觀察狀態不變。這裡的泛型說明另假設析構不拋例外，實際配置器、容量乘二的整數上界與清理流程須由實作處理。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.2　從另一端取出：佇列的操作契約

- 抽象序列同樣為 $[a_0,\ldots,a_{n-1}]$。enqueue(x) 加在末端，dequeue() 移除開頭。
- 若每次移除開頭都把陣列左移，單次 dequeue 為 $\Theta(n)$。
- 改以環狀陣列儲存，可避免為保留邏輯順序而搬動所有剩餘元素。
- FIFO 描述的是回傳次序，不保證多執行緒安全，也不等於每個工作具有相同執行時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

環狀陣列：邏輯順序與實體槽位分開

- 令容量 `C>0`，儲存 `head` 和元素數 `size`。第 `k` 個邏輯元素在 `(head+k) mod C`。
- 空條件 `size=0`，滿條件 `size=C`。不需浪費一格區分 `head==tail` 的歧義。
- enqueue 寫入 `(head+size) mod C` 後增加 `size`。dequeue 讀 `head`，使 `head=(head+1) mod C`，再減少 `size`。
- 每次操作最壞 `O(1)`，前提是固定容量及常數時間元素移動。擴容時需依邏輯順序複製，單次為 `O(n)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

索引繞回，值的先後次序不變

容量為 4，依序加入 `A,B,C`，移除兩次，再加入 `D,E,F`。

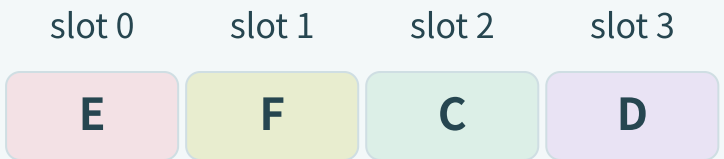
狀態	head	size	邏輯序列
加入 ABC	0	3	A,B,C
移除兩次	2	1	C
加入 D,E	2	3	C,D,E
加入 F	2	4	C,D,E,F

最後實體位置 0、1、2、3 依序存 `E,F,C,D`。

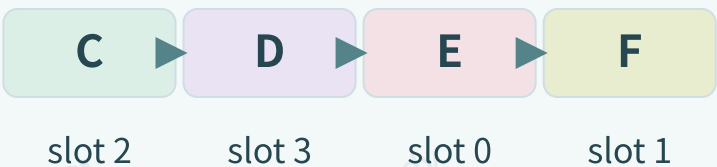
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

值 C 在實體槽2，卻是邏輯上的第一個

實體儲存：head=2，size=4，C=4



邏輯次序：從 head 開始讀



slot(k) = (head+k) mod C

同一個值C：實體槽2 ↔ 邏輯位置0；不是搬移值來整理FIFO

head=2、size=4；依序讀取槽 2→3→0→1，得到 C→D→E→F。滿佇列不能再 enqueue；本章不覆寫最舊值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

固定容量：先驗證，再改狀態

以 head、size 區分空滿，容量 C>0。以下是固定容量版本；try_enqueue 遇滿回報失敗，不覆寫最舊值。回傳值的移動成本與例外政策由元素型別契約決定。

```
slot(k):
    require 0<=k<C
    return head+k if k<C-head else k-(C-head)
try_enqueue(x):
    if size==C: return full
    construct_at(A+slot(size),x)
    size=size+1; return success
```

https://github.com/xinhi

出列：回傳值、銷毀物件、推進 head

```
front(): require size>0; return reference(A[head])
dequeue():
    require size>0
    value=read_or_move(A[head])
    destroy_at(A+head)
    head=0 if head==C-1 else head+1
    size=size-1
    return value
```

先假設值操作成功且析構不拋例外。泛型回傳的 move 若可能失敗，需另訂例外契約，不能默認回傳失敗一定可還原。

用抽象映射核對入列與出列

分支式 slot 避免先算 head+k 溢位。呼叫時先確認未滿，故 k=size<C；出列最後一項後 head 可停在任一合法槽，不需強制歸零。正確性由 `(head+k) mod C` 的抽象映射驗證：入列只新增第 size 個邏輯值，出列後原第 k+1 個值成為新第 k 個值。兩者固定容量最壞 Θ(1)，並非只靠平均。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

擴容必須依邏輯順序重新排列

從 physical=[E,F,C,D]、head=2、size=4、C=4 擴成 C′=8。若只把四格複製至新陣列前四格卻沿用 head=2，新取模會把原應回繞的 E 指向尚未建構的槽4，立即破壞表示。

邏輯 k	舊槽 (2+k) mod 4	值	新槽
0	2	C	0
1	3	D	1
2	0	E	2
3	1	F	3

Lin Cing https://github.com/xinhi6716/computer-science-self-study

搬移成功後，才提交新儲存

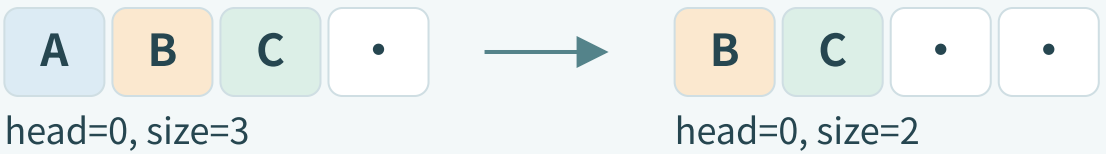
```
allocate new capacity C'
for k=0..size-1: construct new[k] from old[slot(k)]
commit new storage; head=0; C=C'; destroy/release old storage
```

提交後邏輯值仍 C,D,E,F；append G 寫新槽4。擴容單次 Θ(size)，倍增且僅在滿時成長，從空開始入列可按相同幾何級數得攤銷 Θ(1)。固定容量即時系統若要求最壞常數，不能把這項擴容悄悄包含在內。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

空、滿與搬移成本，分別判斷

線性左移：搬B與C



環狀：B與C留在原槽



C=4；空與滿可有相同head，以size=0或4區分

固定容量4時，空與滿可能有相同 head；必須用 size 或另一套明確約定區分。線性左移版出列要搬移所有剩餘值，環狀版只推進 head 並銷毀原首物件，兩者保持相同 FIFO 抽象值，但成本不同。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

環狀佇列的三種空滿慣例

令陣列長度 $C \geq 2$。本章的 `head,size` 使用全部 C 格：`size=0` 為空，`size=C` 為滿。另一種 front/rear 慣例讓 front 位於首元素之前，預留一格：`front=rear` 為空，`(rear+1) mod C=front` 為滿，最多保存 $C-1$ 個值。

若仍令 front 位於首元素之前，也可加一個 tag 區分 `front=rear` 的兩種狀態：tag=false 表示空，tag=true 表示滿，因此可使用全部 C 格。只在兩索引相等時依 tag 判定空滿；非空時首元素是 `(front+1) mod C`，對應 head/size 的 `head`。不同實作必須先寫出完整更新與失效條件，不能混用空滿測試。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Tag 版本：何時更新狀態？

初始 `front=rear=0, tag=false`。假設元素的建構、讀取與銷毀都成功：

```
enqueue(x):
    if front==rear and tag: return full
    rear=(rear+1) mod C; store x at rear
    if rear==front: tag=true
dequeue():
    if front==rear and not tag: return empty
    front=(front+1) mod C; take and remove A[front]
    if front==rear: tag=false
    return value
```

當 `front!=rear`，tag 可保留先前值，不參與空滿判定。這是固定容量的抽象步驟；泛型物件若會丟例外，必須另行安排提交順序，不能直接宣稱失敗時狀態不變。

3.3　兩個堆疊，合成一個 FIFO

令 I 保存新入列值、O 保存待出列值；兩個序列都由底寫到頂。

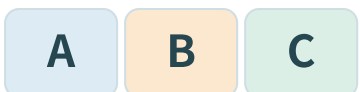
```
enqueue(x): I.push(x)
dequeue():
    if O.empty():
        while not I.empty(): O.push(I.pop())
    require not O.empty()
    return O.pop()
```

https://github.com/xinhi6716/computer-s

舊值在 O，新值在 I，不能插隊

enqueue A,B,C

I bottom → top



O bottom → top

∅

FIFO: A,B,C

dequeue → A

I bottom → top

∅

O bottom → top



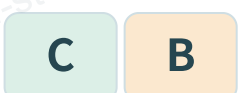
FIFO: B,C

enqueue D

I bottom → top



O bottom → top



FIFO: B,C,D

抽象 FIFO 是 `reverse(O) + I`。**只有 O 空時才搬移**；O 還有舊值時，I 的新值不能越過它們。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一次搬很多，為何仍可攤銷常數？

先把每次 stack push/pop 的成本定為1，底層使用最壞常數的節點堆疊；不把配置器延遲或例外納入此抽象模型。取勢能 $\Phi = 2|I|$，初始為0。

操作	實際成本 c	ΔΦ	攤銷成本 c+ΔΦ
enqueue	1	+2	3
O 非空時 dequeue	1	0	1
O 空，搬 k≥1 值後 dequeue	2k+1	−2k	1

由 $\sum c = \sum \hat{c} + \Phi_0 - \Phi_{final} \le \sum \hat{c}$，m 次成功操作總成本 O(m)。空佇列的失敗檢查另為常數。

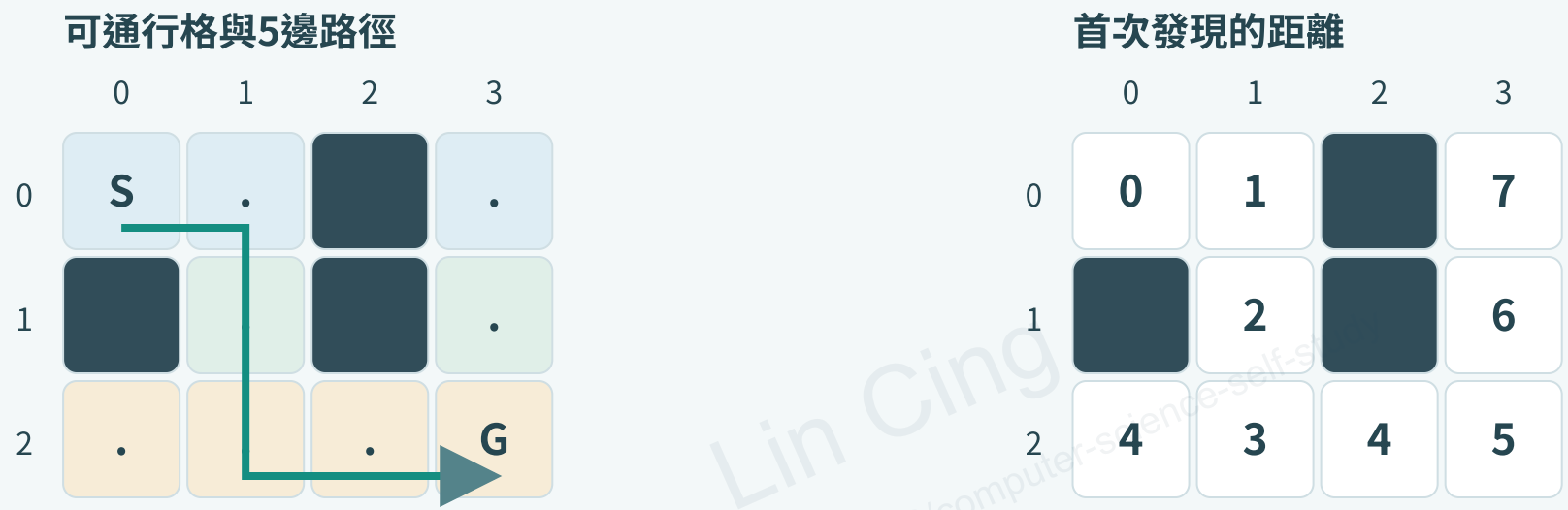
這不是「最壞每次 O(1)」：某次 dequeue 可以搬 Θ(n) 個值。若底層 stack 也會動態擴容，須再加入它的攤銷成本，不能把兩層成本都略去。

3.4　迷宮是圖：先固定可走方向

- 迷宮格子是頂點，相鄰可通行格之間有邊。此例允許上下左右四向移動，不允許斜走。
- 搜尋狀態至少包含所在格、已訪問資訊，以及重建路徑所需的 predecessor。
- DFS 可用堆疊，找到一條路徑但通常不保證步數最少。
- 若所有移動成本都是 1，BFS 使用佇列，首次到達出口的層數就是最短步數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一張迷宮，對照路徑與距離



座標為 (row,column)，左上為 (0,0)，列與欄都從0起算；只允許右、下、左、上，移動成本均為1。深色為牆，折線為 S 到 G 的5邊路徑。

同一張迷宮，對照路徑與距離 （續）

此圖恰好只有一條 S–G 簡單路徑，所以 DFS 與 BFS 在本例得到相同長度；**不能拿本例證明 DFS 一般也最短**。後面的五點反例會分開比較兩者。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFS 框架：保存回來後要繼續的位置

若只把目前格放堆疊而不記下一方向，回溯後可能重做鄰居掃描。框架 `(u,next_direction)` 保存遞迴的程式位置，visited 在首次推入時設定。

```
visited[start]=true; parent[start]=none; push((start,0))
while stack 非空:
    (u,i)=top()
    if u==goal: return parent path
    if i==4: pop(); continue
    top().next_direction=i+1
    v=neighbor(u,i)                  // 右、下、左、上
    if v 在界內且非牆且 not visited[v]:
        visited[v]=true; parent[v]=u; push((v,0))
return unreachable
```

DFS 終止與 parent 路徑的理由

本章格圖在首次走到 (2,1) 時，右鄰 (2,2) 會先被推入，再往右找到 (2,3)=G；路徑共5邊。若不在首次抵達 G 時提早停止，完整 DFS 發現次序為 (0,0),(0,1),(1,1),(2,1),(2,2),(2,3),(1,3),(0,3),(2,0)。每一框架最多嘗試四方向；有限格數且只首次推入，保證終止。parent 邊沿發現時間嚴格回退，因此不成環並能還原合法路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

回溯不是重來：只彈出走完的框架

框架=(row,column,next_direction)；方向編號0右、1下、2左、3上

死路處：上層也已試完

top → pop

(0,3,4)

(1,3,4)

(2,3,4)

(2,2,1)

(2,1,1)

僅畫共同前綴之上的框架；底下仍保留S、(0,1)、(1,1)

退回後：從下方繼續

(2,1,1)

next = 1

Lin Cing https://github.com/xinhi6716/computer-science-self-study

為展示真正的 pop，本頁暫不在 G 停止，繼續走到右上死路 (0,3)。它的四方向都處理完後，彈出 `(0,3,4)`；下一個 top 是 `(1,3,4)`，也已無方向可試。

回溯不是重來：只彈出走完的框架 (續)

再回到 (2,3,4) 後繼續退棧，最後恢復 (2,1,1)。這裡的1表示**右方已試過，下一個從下方開始**，接著才會發現左方 (2,0)。visited 不會因退棧而清除，因為本演算法只要求可達性，不是在列舉所有簡單路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BFS：發現時就設定距離與 parent

每個格子最多入列一次。先標記再 enqueue，可避免兩個同層前驅把同一格重複排入。

```
dist[start]=0; parent[start]=none; enqueue(start)
while queue 非空:
    u=dequeue()
    for v in 四向合法鄰居(u):
        if v 尚未發現:
            dist[v]=dist[u]+1; parent[v]=u; enqueue(v)
```

把 dist 的「未定義」當作尚未發現標記。parent 在首次發現時固定；完整距離圖見前頁。若只要找 G，可在首次發現 G 後還原路徑，不必完成整張圖。

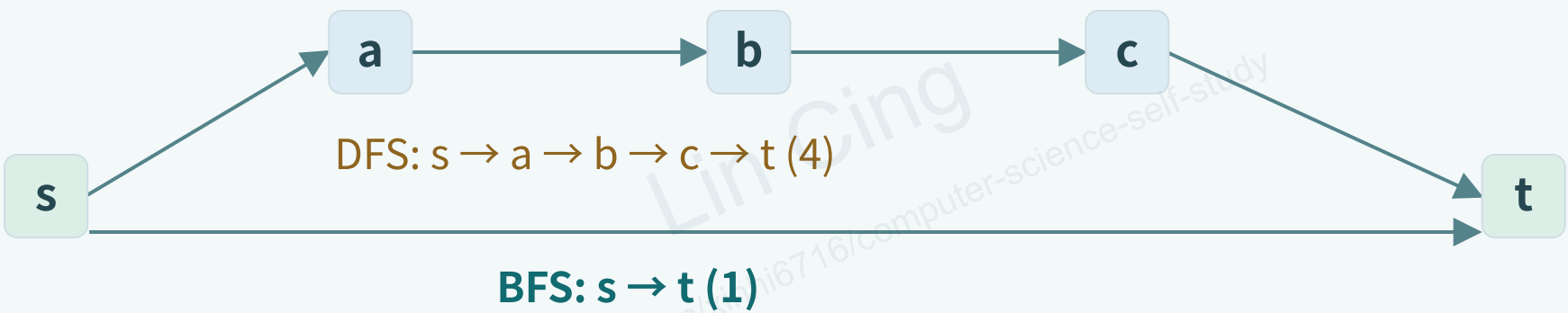
https://github.com/xinhi6716/computer-science-self-study

BFS 逐次出列：同層先處理，才輪到下一層

出列格	新發現格（距離）	出列後佇列
(0,0)	(0,1):1	[(0,1)]
(0,1)	(1,1):2	[(1,1)]
(1,1)	(2,1):3	[(2,1)]
(2,1)	(2,2):4、(2,0):4	[(2,2),(2,0)]
(2,2)	(2,3):5	[(2,0),(2,3)]
(2,0)	無	[(2,3)]
(2,3)	(1,3):6	[(1,3)]
(1,3)	(0,3):7	[(0,3)]
(0,3)	無	[]

從 parent 還原，再證明最短性

G 的 parent 逆追 (2,2)、(2,1)、(1,1)、(0,1)、(0,0)，反轉得5邊路徑。當距離 d 的格被出列時，佇列只可能還有距離 d 或 d+1 的格；新發現格的距離為 d+1。若存在更短路，其前驅所在的更小層早已被處理，與「尚未發現」矛盾。



固定s的鄰居次序為a、t；箭頭是有向邊，每條成本1

反例另固定 s 的鄰居順序 a、t；DFS 先走長路4邊，BFS 在處理 s 時便發現1邊的 t。

可達性、最短性與成本，要各自證明

- 不變量為每個被探索的格都從起點可達，predecessor 邊連回較早發現的格。
- DFS 若探索所有合法鄰居且沒有未處理的堆疊框架，就已涵蓋所有可達格；出口未被發現則不可達。
- 有 V 個可通行格、E 條相鄰邊時，鄰接串列模型的 DFS/BFS 為 $O(V+E)$，標記與 predecessor 空間 $O(V)$。
- 四鄰格子中 $E=O(V)$，所以是線性成本。若每步複製整條路徑，實作可能超出此界。

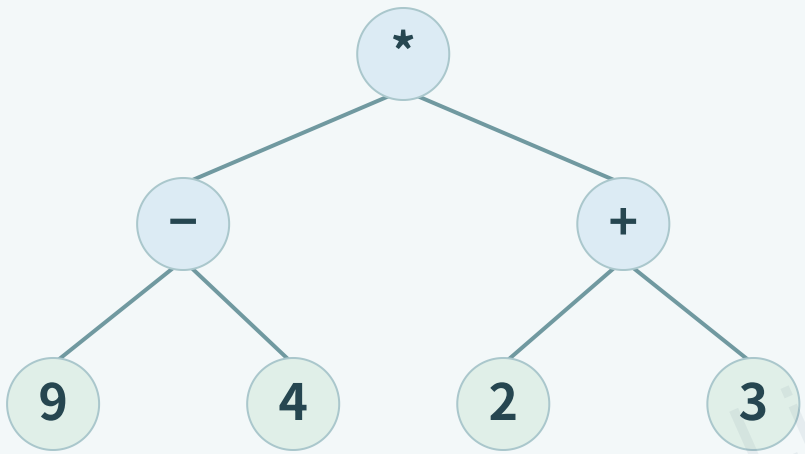
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.5　運算樹決定左右，寫法決定掃描順序

- 中序 `a+b*c`、前序 `+ a * b c`、後序 `a b c * +` 代表相同的運算樹。
- 本章先限定二元運算子、整數 token、括號與明訂優先序。負數 token 與一元負號是不同的語法設計。
- 中序需要優先序與結合性。`a-b-c` 預設左結合，代表 `(a-b)-c`。
- 空白不能可靠地切分所有實際程式語言。完整編譯器需先詞法分析，再處理語法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一棵樹，三種讀法



後序求值：先pop右子式值，再pop左子式值

前序：根 → 左 → 右

`* − 9 4 + 2 3`

後序：左 → 右 → 根

`9 4 − 2 3 + *`

$(9-4) \times (2+3) = 5 \times 5 = 25$

根 * 的左子式是 9−4，右子式是 2+3。前序先根再左、右；後序先左、右再根。中序若要完整保留這棵樹，必須寫成 (9−4)*(2+3)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

後序求值：先 pop 的是右運算元

1. 從左到右掃描 token。數值直接 push。
2. 遇到二元運算子，先 pop 右運算元 `b`，再 pop 左運算元 `a`，計算 `a op b` 後 push。
3. 若不足兩個值即拒絕。掃描完成後堆疊必須恰好剩一個值。

原創例子 `8 3 2 * -` 的堆疊依序為 `[8]`、`[8,3]`、`[8,3,2]`、`[8,6]`、`[2]`。除法還需檢查零除與算術範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前序反向掃描，左右角色也要跟著看

- 前序求值可由右到左掃描。遇到運算子時，第一次 pop 是左運算元，第二次是右運算元。
- 原例 `- 8 * 3 2` 由右向左算出 `3*2=6`，最後 `8-6=2`。
- 每個 token 至多 push、pop 固定次數，所以中序轉後序及前後序求值皆為 `O(t)`，`t` 為 token 數。
- 輔助堆疊最壞 `O(t)`。這裡把固定範圍算術視為常數時間，任意精度數值另計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把「何時彈出」寫成一個判斷

本節只含非負整數 token、二元 `+ - * / ^` 與括號。`^` 表示**右結合冪次**，不是 C++ 的 XOR；其餘左結合。負號不是一元負號，負數 token 也不在本節輸入文法內。

類別	precedence	associativity
`+`、`-`	1	left
`*`、`/`	2	left
`^`	3	right

```
must_pop(top, incoming):
    return top != '(' and
       (prec(top) > prec(incoming) or
        (prec(top) == prec(incoming) and incoming is left-associative))
```

比較的是 incoming 的結合性。 遇括號另作配對處理，不把括號當普通二元運算子。

中序轉後序：逐 token 維護兩個容器

output 保存已確定的後序片段；ops 保存待決運算子。need_operand 防止兩個數值相鄰、空括號與缺少右運算元。

```
output = []; ops = []; need_operand = true
for token in tokens:
    if token 是數值:
        require need_operand; output.append(token); need_operand=false
    else if token == '(':
        require need_operand; ops.push('(')
    else if token == ')': close_group()
    else if token 是二元運算子 op:
        require not need_operand
        while ops is not empty and must_pop(ops.top,op):
            output.append(ops.pop())
        ops.push(op); need_operand=true
    else: reject
```

右括號：只清空當前括號內的待決運算

```
close_group():
    require not need_operand
    while ops 非空且 ops.top != '(': output.append(ops.pop())
    require ops 非空; ops.pop()      // 丟棄配對左括號
    need_operand=false
```

這個 helper 共用 output、ops、need_operand。成功後 need_operand=false；空括號、缺少右運算元或無左括號可配時，都拒絕。括號本身不會輸出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

掃描完，不代表轉換已完成

接續上一頁的同一個 output、ops 與 need_operand：

```
require not need_operand                    // EOF，拒絕空式或缺最後運算元
while ops 非空:
    require ops.top != '('                  // 未配對左括號，拒絕
    output.append(ops.pop())
return output
```

每個 require 失敗都拒絕整個輸入。即使已產生部分 output，也不能把它當成轉換成功的結果。while 不能改成 if：一個新運算子可能要求連續彈出多個舊運算子。

https://github.com/xinhi6716/computer-science-self-study

最小例子：9−4 也需要 EOF

9-4

讀入 token	運算子堆疊（底→頂）	已輸出
9	∅	9
-	-	9
4	-	9 4
EOF	∅	9 4 -

EOF 前只有 9 4，不是完整後序式。EOF 彈出 – 得 9 4 –，值為5；前序為 – 9 4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

括號：先完成內部子式

`(9-4)*(2+3)`

讀入 token	運算子堆疊（底→頂）	已輸出
(	(	∅
9	(	9
-	(-	9
4	(-	9 4
)	∅	9 4 -

右括號使第一個子式 9−4 完成，輸出 9 4 −；這時 ops 為空，接著讀乘號與第二個左括號。

接續第二個括號，最後清空乘號

(9-4)*(2+3)

讀入 token	運算子堆疊（底→頂）	已輸出
*	*	9 4 -
(	* (	9 4 -
2	* (	9 4 - 2
+	* (+	9 4 - 2
3	* (+	9 4 - 2 3
)	*	9 4 - 2 3 +
EOF	∅	9 4 - 2 3 + *

後序求值堆疊為 [9]→[9,4]→[5]→[5,2]→[5,2,3]→[5,5]→[25]。前序 * - 9 4 + 2 3 也算得25。

連續彈出與右結合，分別追蹤

```
9-4*2/1+3
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

連續彈出與右結合，分別追蹤 （續）

讀入 token	運算子堆疊（底→頂）	已輸出
9	∅	9
-	-	9
4	-	9 4
*	- *	9 4
2	- *	9 4 2
/	- /	9 4 2 *
1	- /	9 4 2 * 1
+	+	9 4 2 * 1 / -
3	+	9 4 2 * 1 / - 3
EOF	∅	9 4 2 * 1 / - 3 +

同級要不要彈出？看新運算子的結合性

讀 `+` 時先彈 `/`，再彈同級左結合 `-`；只彈一次會錯。正確值 `9-(4×2)/1+3=4`。右結合 `2^3^2` 則在第二個 `^` 不彈同級，後序為 `2 3 2 ^ ^`，值512，不是64。

右結合：2^(3^2)=2^9=512；左結合：(2^3)^2=8^2=64。這是兩棵不同的運算樹，不是求值順序可以任意替換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

拒絕非法語法，也檢查算術

輸入	失敗時刻	拒絕理由
`(9-4`	EOF	彈 − 後仍有 (，左括號未配對
`9-4)`	讀)	彈 − 後無 (可配
`9-`	EOF	need_operand 仍為真
`9 4`	讀第二個數值	缺二元運算子
`()`	讀)	括號內缺運算元

不可用堆疊值數量猜一元或二元語法。求值另拒絕除零與固定整數溢位。

Lin Cing
https://github.com/xinhi6716/computer-science-s

不變量與線性時間，為何成立？

把尚未配對的左括號視為堆疊分段邊界。每一分段內，已堆疊運算子由底到頂的優先序非遞減；相等優先序只有等待右結合運算時可同時保留。輸出是已確定子運算式的後序排列；堆疊運算子尚在等待右運算元或更高優先子式。

讀入新 op 時反覆 pop，恰好把語法上應先完成的運算子輸出。右括號把該括號內全部待辦運算輸出，保留外層的運算子；EOF 則不再有未讀運算元，必須將全部待辦輸出。若還有左括號，文法就沒有合法配對，不能當運算子輸出。

每個運算子至多推入一次、彈出一次，括號亦如此。雖然某一 token 觸發多次 pop，整段 t 個 token 仍 O(t) 時間、O(t) 空間。這是跨 token 的總量論證，不是宣稱每次迴圈皆最壞常數。剖析負整數 token、一元運算、函式呼叫與浮點特殊值，需擴充文法及對應正確性，不在此二元版默認支援。

括號配對：堆疊記住尚未關閉的種類

輸入只含 `()[]{}`。掃描左括號時 push；掃描右括號時，若堆疊為空，或 top 不是相應的左括號，立即拒絕；否則 pop。掃描結束時，只有堆疊為空才接受。若允許其他字元，必須另訂是否忽略或拒絕，不能默默假設。



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼只比較數量不夠？

`([)]` 中各種左、右括號數量相同，但讀到第三個字元 `)` 時，堆疊自底至頂為 `(`、`[`，頂端期待 `]`，因此拒絕。相對地，`([])` 的狀態依序為 `[ ( ]`、`[ (, [ ]`、`[ ( ]`、`[]`，最後接受。

不變量：掃描過的前綴若尚未拒絕，堆疊由底至頂保存所有未配對左括號，且頂端是下一個必須先關閉的括號。每個符號最多 push 或 pop 一次，時間 $O(n)$、額外空間 $O(n)$；空字串在此契約下接受。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

已知長度的迴文：先反轉前半段

輸入是長度已知的符號序列 S。先把前 $\lfloor n/2 \rfloor$ 個符號依序 push；若 n 為奇數，跳過正中間的一個符號；再由左至右掃描後半段，每次與 top 比較並 pop。遇到不等立即拒絕，全部相等才接受。

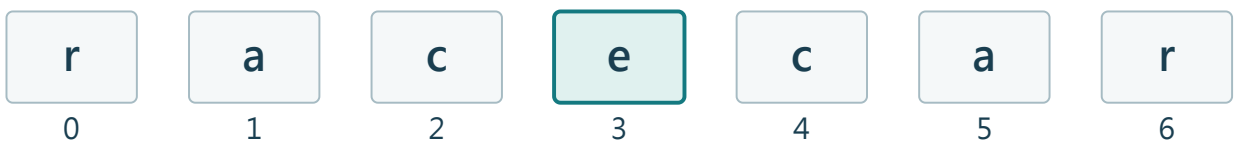
以 `racecar` 為例：先 push `r,a,c`，跳過 `e`，再依序把 `c,a,r` 與堆疊頂端比較。空序列和單一符號都接受。比較單位必須先指定：byte、Unicode 碼點與使用者感知的字形簇可能有不同結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

已知長度的迴文：先反轉前半段 （續）

迴文：前半入堆疊，後半逐一比較

racecar：長度 7，跳過中央 e



前半 push r,a,c；堆疊底→頂為 [r,a,c]。

跳過 e；依序比較 c、a、r，三次都相同。

每次比較後 pop；掃描完成且全相同，接受。

空序列與單一符號也接受；已知長度是此方法的前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

迴文正確性與成本

推入前半段後，top 是中央左邊第一個符號；第 k 次 pop 比較的是相對中央對稱的兩個位置。每一對都相等，恰好等價於序列與其反轉相同。時間 $O(n)$，堆疊最多 $\lfloor n/2 \rfloor$ 個符號。

這個版本事先知道長度並能掃描相應位置。若是單向串流且事前不知道長度，不能在讀到中間前憑空知道何時停止 push；需要不同的輸入假設或演算法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

判定 $a^n b^n$：數量相同，順序也要對

定義語言 $L = \{a^n b^n \mid n \geq 1\}$，輸入字母表只含 a,b。先讀一個以上的 a，每個 push 一個標記；讀到第一個 b 後進入第二階段，每個 b pop 一個標記。第二階段再看到 a、遇到其他符號或空堆疊時讀到 b 都拒絕。結束時必須見過 a 與 b，且堆疊為空。

aabb 接受；aab 留下一個標記而拒絕；abb 第二個 b 無標記可 pop；abab 在第二階段又讀到 a 而拒絕。空字串不屬於此處的 L；若改定義 $n \geq 0$，才應接受空字串。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.6　堆疊排列：不是所有輸出次序都可行

輸入固定依序為 1、2、3，只能把下一個輸入 push，或把當前 top pop 到輸出；禁止直接越過堆疊。

目標	必要操作	結果
2,1,3	push1、push2、pop2、pop1、push3、pop3	可行；最大深度2
3,1,2	為先輸出3，必須 push1、push2、push3、pop3	不可行：top=2 擋住下一個1

一般判定：依目標值 x，若 top≠x，就持續推入尚未讀取的輸入；若輸入耗盡仍 top≠x，立即拒絕，否則 pop x。**正確性來自操作被迫性**：top 不是 x 時不能先輸出其他值；若尚可產生 x，唯一前進方式是 push 下一個輸入。每個值至多進出一次，時間 O(n)、空間 O(n)。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

為何 n=3 有5種，而不是6種？

令 push 為 U、pop 為 D。合法操作有 n 個 U、n 個 D；每一前綴皆滿足 U 數≥D 數，最後兩者相等。這正是合法括號序列（U 對應左括號）。

把第一次 push 與讓它離棧的 pop 配成 `U A D B`。A 內若有 k 次 push，B 就有 n−1−k 次；兩者各自合法，分解也唯一。因此

$$C_0 = 1, \qquad C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}.$$

依序算得 C_1=1、C_2=2、C_3=2+1+2=5、C_4=5+2+2+5=14。固定互異輸入時，輸出唯一決定操作序列（前頁的被迫性），因此也在計算不同的可行輸出，而不只是數操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Catalan 公式：扣掉會對空堆疊 pop 的路徑

不限制前綴時，共有 $\binom{2n}{n}$ 種 U/D 排列。對任何非法路徑，把「第一次下降到−1」以前的 U/D 全互換；原前綴多一個 D，交換後整條路徑共有 n+1 個 U、n−1 個 D。

反向則取這類路徑第一次升到+1的前綴再交換，恰好還原一條非法路徑，所以是一一對應。非法路徑數為 $\binom{2n}{n-1}$。因此

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}.$$

n=0 另有一個空操作序列。互異值、固定入列順序與無旁路，是這裡把 Catalan 數用於堆疊輸出的前提；重複值的不同輸出序列數不能直接套用。

參考資料與署名

Lin Cing，《資料結構》手寫筆記；C++ 標準草案 stack、queue；MIT 6.006，Lecture Notes 9–10。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study